

Project conclusion report - large teams

BTH, May 2026

Authors: Karam Matar, kamt23@student.bth.se

Course: PA1449 VT26 lp34 Avancerat programvaruprojekt i grupp

Team name: Team Softhouse / Team Temp App



Summary

In this report I look back at my work in the Softhouse project, where our team built Team Temp App, a survey system for creating, editing, scheduling, answering, comparing, and following up internal surveys. Softhouse had previously relied on Google Forms and similar generic tools, but wanted something better suited to its visual identity and company culture. My work moved between communication and technical tasks, including customer and HoD coordination, Cypress E2E testing, backend validation, vulnerability handling, merge conflict resolution, PR reviews, and feature work such as configurable scale questions and role handling after branch changes. The report reflects on how the team used agile artifacts such as sprint planning, backlog prioritization, demos, and Definition of Done, and how our process compared with the Scrum Guide. It also covers quality assurance and testing across multiple levels, secure delivery through pipelines and version control, customer relation and feedback handling, and sustainability thinking applied to reminders, AI usage, and privacy. I end with personal learning and lessons about task ownership, test reliability, communication, and what I would do differently in a future team project.

1 Project Information

1.1 Project Assignment

The assignment was to build a survey platform for Softhouse. Softhouse had previously used Google Forms and other generic form tools for internal employee surveys, but wanted something more tailored to its own visual identity, workflow, and company culture. Administrators needed a way to create, edit, schedule, compare, and manage surveys, users needed a simple way to answer them, and the organization needed clearer follow-up and higher response rates than a generic tool could provide.

The goal went beyond replacing a form tool. The team worked toward one product for survey creation, editing, scheduling, survey answering, reminders, analytics, comparison of results, reusable survey formats, anonymous answers, gamification, and access control based on user roles. Gamification and a rewards system were introduced as one part of this broader goal: to make survey participation more motivating and encourage employees to answer, rather than treating surveys as a chore. The product target users were company admins, branch admins, and colleagues. The technical role model used `COLLEAGUE`, `BRANCH_ADMIN`, and `COMPANY_ADMIN`. A mobile app was also created so that colleagues could answer surveys from their phones, but this did not introduce a separate user role. Earlier planning material sometimes used super admin as a planning term for broader admin functionality.

For me, the project became a survey lifecycle rather than just a set of screens: an admin creates or schedules a survey, selects branches or users, optionally uses AI or templates, sends reminders, users answer, and admins follow results through dashboards or Qlik. My own contribution involved both communication and technical work. I helped turn customer feedback into team work, and I contributed to Cypress E2E testing and test maintenance, backend validation, vulnerability feedback handling and Jira task creation, test environment debugging, merge conflict resolution, pull request reviews, branch and user handling, configurable scale questions, demo preparation, and the admin-to-user role change after branch switch.

1.2 Process and Method

The Scrum Guide defines sprints as events of a fixed length where work is planned, executed, reviewed, and adapted, building on the three empirical pillars of transparency, inspection, and adaptation [1]. Our process followed this shape. The team worked in two-week sprints that

aligned with the course schedule. Each sprint began with a planning session where customer needs, demo value, and technical blockers were weighed. The team then implemented, tested, and reviewed work through pull requests and pipeline checks. Each sprint ended with a demo that gave the customer and HoD a chance to inspect the increment.

Standups happened throughout the sprints, but they were informal: usually daily or frequent on-site syncs between the team members who were present at Softhouse, rather than formal Scrum Daily Scrums. The team also held brief retrospective discussions after sprints or demos. These were used to reflect on what worked, what did not, and how to plan the upcoming sprint, but they were not always run as formal textbook retrospectives. Our sprint commitments were shaped more by customer priorities and the S to C ranking of user stories than by velocity based on story points. The consequence was flexibility but also less predictability when customer feedback arrived late, as happened in week 17 when Casper’s feedback came too close to the demo to act on fully.

Jira was used for task planning, Bitbucket for branches and pull requests, and Bitbucket Pipelines for delivery checks [3, 4]. My role in this process was often to keep communication moving: I booked customer and HoD meetings, prepared demo material, followed up decisions through Discord, and shifted to testing or implementation work when the sprint needed it.

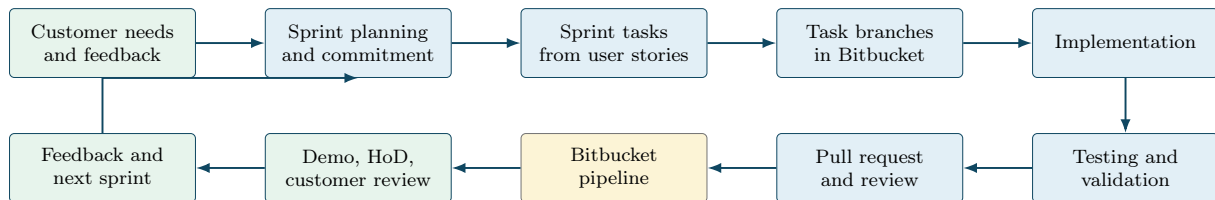


Figure 1: The process used during the project.

The diagram shows why demos were important. They were checkpoints where the customer, HoD, and team could see whether the commitment still matched the product value. Feedback was documented and moved into later Jira tasks or sprint commitments. Compared with strict Scrum, our work was adapted to a student project with external access issues, learning time, and changing customer feedback. We had sprint commitments, demos, Jira tasks, and reviews, but some artifacts were less formal than Scrum recommends. The consequence was flexibility but also less predictability when tasks were too large or when feedback arrived late.

1.3 System Architecture

The system was built around three client surfaces (an admin portal, a user portal, and a mobile app), all communicating with the same Bun/Express backend through a REST API. The backend used Prisma as its ORM and MongoDB as the database. Authentication relied on Better Auth with Google OAuth for single sign on, combined with middleware that enforces access based on roles (`requireAuth`, `requireAdmin`, `requireCompanyAdmin`). Around the core survey lifecycle, the system integrated Cloudinary for image uploads and Slack for notifications and reminders, an AI service for survey suggestions and translation, and Qlik for analytics dashboards embedded in the admin portal.

The diagram is intentionally high level. The dashed arrow marks an indirect connection: Qlik dashboards are embedded as iframes in the admin portal rather than accessed through the backend API. All other arrows represent direct runtime communication. My own work mainly touched the backend validation layer, survey access and fetching logic, Cypress-tested admin and user flows, and role-related behavior around users and branches.

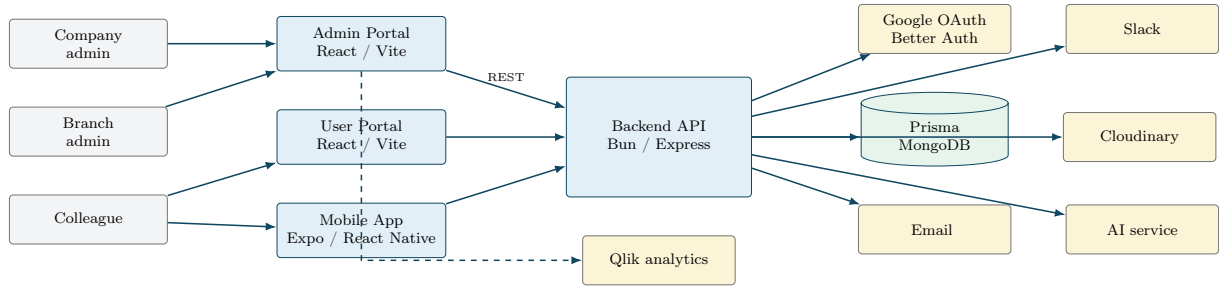


Figure 2: High-level system architecture.

1.4 Time, WBS, PBS, Requirements, and Meetings

My final recorded and estimated project time was approximately 185 hours. This time was distributed across communication and coordination, Cypress testing and QA, backend validation and security work, demo preparation, integration and support work, and PR reviews, merge conflict support, and debugging. The team did not use story points consistently throughout the project, so I use tracked hours, weekly reports, Jira task evidence, and qualitative contribution mapping instead of velocity or story-point analysis. I use the WBS as an activity map rather than as an exact measurement instrument. Individual work is represented through these sources and the qualitative contribution mapping shown in the table below.

WBS category	Team use	My contribution
Project Planning	Vision, roles, backlog, time estimates, sprint goals.	Customer and HoD contact, meeting booking, sprint preparation, communication through Discord, and follow-up notes.
Product Design	Requirements, UX, user flows, architecture, MVP.	Helped clarify requirements in customer meetings, the Cinode-to-Qlik analytics clarification, Slack requirements, gamification ideas, and scale question behavior.
Development	Two week sprints, feature work, bug fixing, demos.	Worked on branch and user handling, survey fetching logic, validation and security fixes, configurable scale questions, and branch admin becoming a regular user after branch change.
Quality Assurance	Unit tests, integration tests, UI tests, secure auth and authorization.	Cypress E2E tests and test maintenance, test updates after UI changes, pipeline and test environment debugging, vulnerability report handling and Jira task creation, backend validation fixes, merge conflict resolution, and PR reviews.
Documentation	Technical documentation, deployment guide, final handoff, and progress reporting.	Wrote progress notes, meeting notes, emails after meetings, demo material, project setup documentation, and helped keep decisions visible to the team.
Project Management/Scrum	Standups, sprint planning, retrospectives/reviews, progress tracking.	Took part in sprint planning, demos, HoD meetings, feedback handling after demos, Jira task creation, and coordination of blockers.

Table 1: Qualitative mapping of my work to WBS categories.

Because story points were not used consistently, the table should be read as a qualitative mapping of contribution rather than a quantitative velocity report.

The PBS and customer commitments show product areas such as user management, authenti-

cation, survey creation, response management, notifications, analytics, and access control. My clearest individual contributions, based on project notes and Jira evidence, were in Cypress E2E tests, branch and user handling, backend validation (answer, cross-survey, comment length), configurable scale questions, vulnerability handling, merge conflict resolution, PR reviews, test environment debugging, and the admin-to-user role change after branch switch. I also assisted other members when they got stuck. Broader areas such as AI, Qlik, Slack, mobile, gamification, and documentation were mainly team deliveries, with my contribution described only where evidence supports it.

Product area	Requirement or customer need	What I contributed	Why it mattered
Survey creation	Admins needed to create and manage surveys reliably.	Worked with Cypress tests and test updates for the survey creation flow.	Helped verify that survey creation still worked after UI changes and before demos.
User and branch handling	Users and admins needed correct access depending on branch and role.	Worked on branch and user handling, including the case where a branch admin becomes a regular user after changing branch.	Helped ensure that users had the correct access level after branch changes.
Survey access and fetching	Different users needed to see the correct surveys based on role, branch, and user context.	Contributed to backend logic for fetching surveys based on role, branch, and user information.	Helped prevent users from seeing the wrong surveys or missing surveys they should receive.
Validation and vulnerability feedback	Submitted survey answers needed to be checked on the server side, not only in the browser.	Worked on answer validation, conditional and cross-survey validation, and comment length limits.	Made the system safer by moving important checks to the backend.
Configurable scale questions	Admins needed more flexibility when creating scale questions.	Worked on configurable scale question behavior and related validation.	Made the survey feature more flexible without assuming one fixed scale for all surveys.

Table 2: PBS and customer requirements mapped to my individual contribution.

The main meetings were customer requirement meetings, commitment meetings, HoD meetings, demos, walkthroughs before demos, and later focused meetings about Qlik, Slack, final delivery, and final BTH demo preparation. I contributed value by booking or following up meetings, asking clarification questions, documenting feedback, presenting sprint or technical parts, and making sure information reached the team.

1.5 Roles, Procedures, and Routines

The team originally discussed rotating roles so that all members could experience different responsibilities. In practice, I kept the communication responsible role throughout the project because it became useful for continuity. Maintaining the same contact person toward the customer and HoD avoided losing context across customer meetings, sprint planning, meeting booking, Discord updates, and follow-up. During one sprint, I also took on Scrum Master-like responsibilities, helping drive sprint planning and task coordination alongside my communication duties. As a fullstack member, I moved between frontend, backend, testing, validation, and support tasks depending on what the sprint needed.

At team level, the routines were Discord communication, informal daily or frequent on-site syncs, Jira task planning, Bitbucket task branches, pull requests, code review before merge, CI checks, demos, and feedback followup. Risk management was mostly practical: identify blockers early, ask Softhouse or HoD when stuck, keep test and demo versions stable, and avoid merging work with known critical bugs. Code standards were discussed through clean code guidance, reviews,

TypeScript practices, and tests.

At individual level, my routine combined communication, Jira followup, coding, Cypress testing, backend validation, debugging, demo preparation, and PR work. I learned that diagrams, notes, and demo walkthroughs are also real work, because they helped the team and customer understand what had been built.

1.6 Project Sustainability

A good sustainability requirement for this product would be: *The system shall support responsible reminders and AI usage by only using them when relevant for the customer, administrator, and recipient.* This fits the product because reminders and AI features can create unnecessary digital noise if they reach users who do not need them.

The team did not have a formal sustainability requirement, but some product decisions already moved in this direction. An admin chooses when to send a reminder, and reminders should only go to relevant users. Users who have already answered, are on parental leave, or are otherwise unavailable should not receive unnecessary reminders. The reminder logic was mainly shaped by customer discussions and the tasks derived from those discussions. The customer did not want reminders to be sent broadly to all users, but only to users who should actually receive them. AI-supported functions were also intended to be used when an administrator actively interacts with them, rather than as a constant background process. These choices were therefore connected to customer-driven product decisions, even though they were not written as formal sustainability requirements.

If sustainability had been formalized as part of the deliveries, the team would have needed clearer requirements for reminder frequency, recipient filtering, AI usage limits, data retention, privacy, and accessibility. In practice, this could have become part of sprint planning and the Definition of Done through acceptance criteria such as checking whether reminders are sent only to relevant recipients and whether AI features run only after an active admin action. Since the project did not include a separate sustainability test suite, these checks should be understood as future QA guidance, manual verification during demos, or possible automated tests in a later version. For the customer, such criteria would have supported a more controlled product with fewer unnecessary notifications, lower risk of survey fatigue, clearer privacy expectations, and more responsible use of AI features.

Sustainability was not central because the assignment focused first on functionality and learning. The team added AI suggestions, Slack reminders, and email notifications based on feature value, but did not formalize sustainability criteria such as recipient filtering or usage limits. For sustainability to become integral, it needs to be part of acceptance criteria, sprint planning, and customer prioritization from the start.

2 Individual Learning and Reflections

2.1 Agile Artifacts

Product backlog. The Scrum Guide describes the product backlog as a single, ordered, evolving list of what is needed to improve the product [1]. The product backlog was maintained in Jira and refined together with Softhouse representatives. Initial backlog work came from customer requirement discussions, where user stories were prioritised and later broken down into sprint tasks. Customer-facing user stories were prioritised using S, A, B, and C tiers, with S as the highest priority. S-tier work covered the most important parts of the product, especially direct survey logic and core trust features such as Softhouse account login, survey summaries, anonymity-related behavior, and central survey flows. Lower tiers represented useful but less

critical work, with C-tier mainly covering convenience or polish features such as templates and light/dark mode. In practice, Jira contained both product-level user stories and sprint-level implementation tasks. Compared with the Scrum Guide’s ideal, the limitation was not that the team lacked a backlog, but that the backlog was not always maintained as one consistently ordered and estimated artifact. Prioritisation was mostly driven by customer value, sprint commitments, demo feedback, and urgent findings rather than by stable velocity data. Additional prioritisation came from “shall/might” decisions during demos, feedback severity, and later from critical security issues: after I filed the vulnerability report in week 15, security-related tasks such as TEMP-281 (no validation of answers on the server) and TEMP-283 (cross survey question injection) became important work for me and a few other team members. At the same time, other team members continued with parts of the original sprint plan, so the team handled urgent feedback and planned feature work in parallel.

Compared with the Scrum Guide’s ideal, our approach was more driven by customer needs and less driven by data. The benefit was that prioritisation stayed close to customer value and could adapt quickly when feedback arrived. The drawback was weaker forecasting: without a single ordered backlog and consistent estimation, the team could not easily predict how much work remained or how scope was growing across sprints. The practical lesson is that backlog quality affects implementation quality: when a Jira item said only “fix validation” without specifying which endpoint or what the expected behaviour was, the developer had to guess, and guesses sometimes led to duplicated work or incomplete fixes.

Estimation. At the start of the project, we did not use planning poker. After the second sprint, the HoD suggested that the team should use planning poker to estimate task effort, and we used it for around two or three sprints. After that, the practice faded and story points were not used consistently. Later planning relied more on customer priorities, the S to C ranking, sprint commitments, perceived complexity, known blockers, and what remained from previous sprints. Compared with the Scrum Guide’s emphasis on refinement and using past performance to forecast [1], our approach was less structured. The consequence was that the team did not have a reliable velocity metric, and sprint commitments were based more on judgement than historical data. My lesson is that estimation practices need to become a routine to be useful. If the team does not actively return to story points during planning and follow-up, they quickly stop supporting forecasting.

Sprint planning. The Scrum Guide states that sprint planning should produce a sprint goal and a plan for delivering selected product backlog items [1]. In our project, the Scrum Master created concrete Jira tasks from the relevant prioritised user stories at the start of each sprint. Sprint planning considered customer needs, demo value, available time, technical blockers, and access to external services such as Slack or Qlik. The outcome was a sprint commitment and a set of Jira tasks that formed the sprint backlog. Team members then generally picked tasks from the top of the sprint list and worked downward. Sometimes two people collaborated on the same task, but Jira only allowed one assignee per task, so Jira ownership does not always reflect the real collaboration accurately.

When planning worked well, as in Sprint 6 where customer feedback tasks (TEMP-298 through TEMP-329) were broken into specific validation and UI fixes, user stories became smaller tasks with clearer expected outcomes. When planning worked less well, especially during the first or second sprint, some task descriptions were too broad or overlapping. This happened between other team members and contributed to duplicated work, declined pull requests, or merge conflict situations. This should be understood mainly as a process problem caused by overlapping task boundaries and unclear ownership, rather than as something caused by one single person. My involvement was mainly to assist when members got stuck and help resolve merge conflicts. Later, when I worked on the configurable scale feature (TEMP-351) in week 18, I also encountered merge conflicts, but those were mainly caused by concurrent changes in related files such as translation

and validation work, rather than by unclear task descriptions.

Vertical cut and sprint backlog. A clear vertical cut appeared in the survey lifecycle: admins create surveys, users answer them, responses are stored, and admins review results. This made sprints executable because the team could divide work across frontend, backend, database, tests, and demo validation while delivering one visible product slice. The sprint backlog was built from the S to C prioritised user stories and from the project plan made at the beginning of the course. For each sprint, the Scrum Master and the team discussed which user stories or product areas should be addressed at that point. The Scrum Master then created Jira tasks derived from those user stories. Tasks that were not completed in one sprint carried over to the next sprint, and new customer requirements, discoveries, demo feedback, or tester feedback could create additional tasks on the board. Examples of concrete tasks include Cypress tests for survey creation, branch and user handling (TEMP-193, TEMP-194), security validation fixes (TEMP-281 to TEMP-283), and scale question improvements (TEMP-351). This gave the team flexibility to react to new information, but because story points and velocity were not used consistently, we could not easily compare expected effort with actual effort or forecast remaining work with confidence. The project would have benefited from stricter task sizing and more consistent estimation follow-up.

Definition of Done. We gradually converged on a practical DoD through project needs rather than by writing a perfect definition at the start. It became clearer when we hit merge conflicts, broken tests, pipeline failures, and demo pressure. I learned from this that a DoD is useful only if it matches the problems the team actually has.

The DoD we gradually converged toward in practice was:

1. Code implemented and tested locally
2. Lint passes (`bun run lint` for affected packages)
3. Build passes (`bun run build`)
4. Unit tests pass (`bun run test`)
5. E2E tests pass (`bun run test:e2e`)
6. Pull request opened with description
7. CI pipeline passes (1 of 1 build)
8. Reviewer merges to main

After some time into the project, this looked like the right kind of DoD because it matched the problems we actually had. It reduced the risk of merging code that only worked on one person's computer, and it made delivery more reliable before demos. Customer and demo satisfaction was still useful feedback, but it was not the same as technical DoD, because the DoD was about whether work was ready to merge and deliver.

Retrospectives. The Scrum Guide defines the Sprint Retrospective as an event where the team inspects how the last sprint went and commits to specific improvements for the next one [1]. We did not always run textbook retrospectives as separate meetings. Instead, the useful reflection happened after demos, during HoD feedback sessions, or in team syncs. One clear example was after Demo 4, when the team discussed that the test environment database was leaking test data into production views, which led to a concrete action: separating Cypress test collections (TEMP-290, TEMP-221). The similarity to Scrum's intent was real because we did inspect and adapt, but the difference was that our action items were not always written down or tracked. The consequence was that some improvements stuck (better demo preparation, clearer Jira tasks) while others faded because no one checked whether the team actually changed its

behavior. For me, the lesson was that informal reflection helps but written action points are what make adaptation reliable.

Demos. Demos were performed as prepared walkthroughs with slides and live system use. In Scrum terms, they were closest to Sprint Reviews because they gave the customer and HoD a chance to inspect the product increment and influence the next work [1]. We wanted to learn whether the customer understood the feature, whether the feature solved a real need, what was confusing, and what should be prioritized next. Demo feedback was used directly in later commitments and Jira tasks. Examples include the clarification that the customer’s analytics need was better represented by Qlik rather than the initially mentioned Cinode, Slack notification needs, better error messages, mobile usability feedback, anonymity concerns, and tester feedback that led to validation and privacy fixes.

Scrum board. The scrum board was based on Jira. It helped us see backlog work, sprint tasks, owners, and status. Atlassian describes a board as a visual way to track work items through workflow columns, and that matches how we used it [3]. It could have been improved by making task descriptions smaller and clearer, keeping ownership more explicit, and using board screenshots or reports more consistently for progress review. See Appendix B for a screenshot of the final Jira board state during the polish and handoff period.

Burndown or burn-up chart. The team did not actively use a burn-down or burn-up chart. Instead, progress was followed through Jira tasks, sprint commitments, demos, HoD follow-ups, progress reporting, and time reporting. Since no burn-down or burn-up chart was actively used in the project, the appendix documents this absence rather than adding a reconstructed chart that was not part of the team’s real process. A burn-up chart could still have added value by showing scope growth across the project, because the product grew from core survey flows into AI, Qlik, Slack, mobile, gamification, and security feedback. The Scrum Guide also notes that burn-ups and burn-downs are useful forecasting practices, but they do not replace empirical inspection [1].

2.2 Solution and Technical Reflections

The solution was structured as a Bun monorepo with three client applications sharing one back-end API. The admin portal (React/Vite) handled survey creation and management, user and branch administration, templates, results, AI features, reminders, media uploads, and Qlik analytics. The user portal (React/Vite) provided survey answering, profile and branch information, rewards, and leaderboards. The mobile app (Expo/React Native) offered the same core survey and profile flows for colleagues on mobile devices. Both web portals used TailwindCSS and Radix UI for styling, React Query for server state, Zustand for client state, and i18next for Swedish/English localization.

The backend was mainly organized into routes, controllers, services, and repositories. Routes defined HTTP endpoints and applied middleware for authentication and role checks. Request validation was handled with Zod schemas before business logic continued through the service layer. Services contained the business logic, and repositories encapsulated Prisma calls to MongoDB. This separation made it possible for different team members to work on the same feature domain without constantly conflicting at the file level, although strict adherence varied across modules.

External integrations connected through the backend. Better Auth handled session management and Google OAuth sign in. Cloudinary stored uploaded images. Nodemailer sent email reminders via SMTP. Slack webhooks and a bot API delivered channel notifications and direct message reminders. An AI service, accessed through the Vercel AI SDK, provided survey suggestions, translations, and comment summaries. Qlik Cloud dashboards were embedded as iframes in the admin portal, with each survey type pointing to a configurable dashboard URL.

Component	Role/function	Interaction	Key tools
Admin Portal	Survey creation/management, users, branches, templates, results, reminders, Qlik, media.	REST calls to backend; embeds Qlik dashboard via iframe.	React, Vite, TailwindCSS, Radix UI.
User Portal	Survey answering, profile, branch info, rewards, leaderboard.	REST calls to backend for surveys, answers, and gamification data.	React, Vite, TailwindCSS, Radix UI.
Mobile App	Survey list and answering for colleagues on mobile.	Same REST API. Authentication via Better Auth Expo plugin.	Expo, React Native.
Backend API	Business logic, validation, auth middleware, and integration orchestration.	Receives requests from all clients; calls database and external services.	Bun, Express, Zod.
Prisma / MongoDB	Data model: users, branches, surveys, questions, responses, reminders, templates.	Backend reads/writes through Prisma Client.	Prisma, MongoDB [5].
Auth middleware	Session management, Google OAuth SSO, access control based on roles.	Middleware checks session and role before protected endpoints.	Better Auth, Google OAuth.
Notifications	Email and Slack reminders and survey notifications.	Backend triggers messages through SMTP and Slack API.	Nodemailer, Slack Web API.
AI service	Survey question suggestions, translation, comment summarization.	Backend calls AI provider via Vercel AI SDK from admin features.	Vercel AI SDK.
Qlik analytics	Trend and comparison dashboards for administrators.	Admin portal embeds configured dashboard URL as iframe.	Qlik Cloud.

Table 3: Main components, interactions, and technologies.

The main technologies used by the team were TypeScript, React, Prisma, and Cypress. These were team-wide technical choices rather than tools I selected individually. TypeScript gave type safety across both frontend and backend, which reduced a class of runtime errors. React and the shared component libraries (Radix UI, React Query) made it feasible to build two separate web portals with consistent patterns [6]. Prisma kept database access structured and made the data model explicit through the Prisma schema. In the MongoDB setup, schema changes were synchronized through Prisma’s schema workflow and `db push` rather than through Prisma Migrate migrations [5]. Cypress gave confidence that important survey flows worked from end to end when the test environment was stable [2].

What worked less well was the test infrastructure surrounding Cypress rather than Cypress itself. Failures often came from database rebuilds clearing test data, login session handling between test runs, and differences between the local and pipeline environments. External integration setup also created friction: Slack required a bot token with specific workspace permissions, Qlik needed dashboard URL configuration per survey type, and the AI provider choice had to be confirmed before the team could finalize the integration. These dependencies meant that some features were blocked not by code difficulty but by access and configuration delays.

If I started again, I would push earlier for a stable test environment with predictable test data and clear cleanup rules, and earlier decisions on external service access.

Documentation was handled through README/setup notes, meeting notes, demo material, and later handoff documentation, but it was not always part of the DoD in early sprints. A better approach would have been to include documentation updates directly in task acceptance criteria, especially for integrations such as Qlik, AI, Slack, and deployment.

2.3 Quality Assurance

Quality work in our sprints meant more than writing tests. It meant checking that the committed feature worked, matched the customer need, survived integration with the rest of the system, and could be shown without known critical bugs. In practice, we tried to protect quality through PRs, code review, pipeline checks, backend unit and integration tests, Cypress E2E tests, manual demo walkthroughs, and customer or tester feedback.

The QA setup combined several layers. Backend unit tests with Bun:test covered service logic, controllers, route logic, and validation. Integration-style tests used Bun:test with mocking and Prisma test data to verify service interactions and authorization. Cypress E2E tests covered important admin and user portal flows, especially survey creation, survey answering, and regression-sensitive browser behavior. The project also used ESLint, TypeScript strict mode, Prettier, Zod validation schemas on both backend and frontend, BetterAuth with role-based access control, and Bitbucket Pipelines for CI/CD. Test routes in the backend supported test login, database cleanup, and controlled test-data handling where needed, which made E2E test isolation possible.

Code reviews were based on pull requests in Bitbucket. The review practice was not perfect, but it gave the team a place to discuss changes before they reached main. The CI/CD pipeline ran lint, backend unit tests, builds, and Cypress E2E tests for PR and main flows [4]. A known limitation was that lint was non-blocking in the pipeline due to existing lint debt, so lint warnings could not serve as a full quality gate. This is an honest limitation worth noting: it meant that code style issues could slip through even when the pipeline passed.

Customer and tester feedback created concrete QA work. Examples included anonymity, broken links, behavior of questions with multiple choices, image display, navigation, dashboard clarity, comment length, editing sent forms, accessibility, role handling, and validation. Before demos, we relied on walkthroughs, testing against the correct version, validation on the server side, and regression checks. This reduced risk, but it did not guarantee that no bugs remained. Quality risks were mainly handled through pull requests, CI checks, Cypress tests where relevant, backend tests, and manual testing before demos. The main limitation was not that the team had a fully unstable test suite, but that automated testing did not cover every part of the product equally. Some flows, especially web-based survey creation and answering, had stronger test support, while other parts still depended more on manual review and demo walkthroughs. This meant that quality assurance depended on both automation and disciplined manual checking.

In my view, the biggest prevention for QA problems would have been smaller tasks, a stable test environment earlier, clearer DoD, and stricter ownership of customer feedback. What I personally learned about QA is that the most valuable checks are the ones closest to the customer's actual use, not the ones with the most coverage.

2.4 Test

The test strategy combined automated and manual testing at different levels, loosely following a testing pyramid [2]. The information we wanted from each level was different. Unit tests with Bun:test gave fast feedback about isolated service logic, controller behavior, and validation rules. Integration-style tests, also using Bun:test with mocking and Prisma test data, gave feedback about authorization, service interactions, route behavior, and error handling. Cypress E2E tests gave feedback that important user flows, such as survey creation, survey answering, branch management, and answer editing, worked through the browser. Manual testing and demo walkthroughs gave feedback about usability, real customer expectations, missing requirements, and issues that automated tests did not express well.

My own work included Cypress E2E test setup and maintenance, writing and refactoring survey creation tests, updating tests after UI changes, login and test environment debugging, and

investigating cases where the pipeline showed failures despite tests passing locally. I also contributed to other testing and QA-related work where the sprint required it. A concrete example: in week 17 I spent roughly three hours with team members fixing Google Cloud access and Slack token configuration in the test environment. That time was necessary but invisible in feature output, and it taught me that test infrastructure is as important as test coverage.

The automation level was partial. Backend tests were automated with Bun:test and E2E with Cypress for web portals. Mobile app automated testing was not implemented, so mobile flows relied on manual testing. Web flows therefore had stronger regression support than mobile flows, and manual checks remained important before each delivery. Our weakness was not Cypress itself but the dependency on stable login, predictable test data, and environment state.

Fault handling came through Jira, tester feedback, vulnerability reports, demo observations, and retesting. Zod schemas enforced validation on both backend and frontend. The CI/CD pipeline sequence was: lint (non-blocking), backend unit tests, parallel builds, then parallel Cypress E2E tests for admin and user portals. A broken build or failing unit test would block E2E execution, keeping the pipeline relatively trustworthy despite lint not being a hard gate.

2.5 Secure Delivery

We tried to keep delivery within time, cost, and quality through sprint commitments, Jira tasks, task branches, pull requests, pipeline checks, demos, and a test environment. We reduced version confusion by working from task branches into PRs, merging reviewed work into the shared branch, checking the pipeline, and walking through the demo version before presenting. This was close to a lightweight CI/CD workflow, although final customer access and test environment stability were harder than in an ideal pipeline. Delivery confidence therefore depended on both technical checks and human demo walkthroughs, not only the pipeline. Documentation was handled through README/setup work, feature documentation tasks, meeting notes, and final handoff discussion.

Feature branches were usually tied to Jira task IDs or clear names, such as feature, bugfix, test, feedback, or security branches. That made it easier to connect work to a task and keep the main branch more stable. The system also used authentication based on roles, protected routes, validation on the server, test route gating, upload type and size restrictions, and careful environment handling. I avoid listing any real secret values here.

Security work became more concrete after tester and vulnerability feedback. In week 15 I created a structured vulnerability report and added it to Jira as tasks TEMP-281 through TEMP-287. The report covered three critical issues: CRIT-01 (no validation of answers on the server, allowing arbitrary values such as -100000 for a 1–10 scale), CRIT-02 (conditional question logic enforced only in the browser), and CRIT-03 (cross survey question injection where a `questionId` from one survey could be submitted inside another). It also included four medium-severity issues such as exposed Firebase UIDs and missing input validation layers. I worked on the main backend fixes for TEMP-281, TEMP-282, and TEMP-283. Because these fixes strengthened the shared server-side validation logic, some related medium-severity issues were also reduced or resolved through the same changes rather than needing completely separate fixes. The exact handling depended on how each issue related to the validation layer. In our project, we treated backend validation as necessary because the vulnerability feedback showed that browser-only checks could be bypassed. Zod schemas on the backend enforced stricter type and range validation for submitted answers, and the fixes moved conditional logic and cross-survey checks to the server side. I cannot guarantee that no bugs remained, but the critical validation issues were addressed before final delivery.

2.6 Customer Relation

I learned who the customer was through the first Softhouse visit, the NDA process, requirement meetings, demos, HoD/customer meetings, and later discussions with Softhouse representatives. Early meetings gave the team a clearer view of why Softhouse wanted something more tailored than Google Forms. As communication responsible, I contacted the customer and HoD, booked meetings, sent information, and kept the team updated through Discord.

What I could bring from the customer into the project was mainly clarification and feedback: requirement discussions, demo feedback, the analytics clarification where early discussions mentioned Cinode but later customer input showed the relevant need was Qlik dashboards, Slack access discussions, tester feedback that became Jira tasks, and the vulnerability report. A specific moment that showed the value of customer contact was the monthly Softhouse presentation in week 18: the team rehearsed timing, presented to the entire company with microphones, and answered questions about the project and AI. The positive reception confirmed that the product had real value beyond our course assignment.

The customer relation could have been improved through earlier access to external services and clearer feedback timing. In week 17, I met with Emma to clarify Slack integration requirements and learned that the real Slack workspace required a Bot Token with specific access rights, information that would have been useful weeks earlier. The lesson is that a communication responsible role is also about identifying and removing access blockers. Next time I would push for external service access earlier and ask which extra feature matters most before the team commits extra scope.

3 Summary and Lessons Learned

Expectations on myself. I expected myself to contribute both technically and communicatively. I did that through customer coordination, Cypress testing, backend validation, vulnerability handling, merge conflict resolution, PR reviews, demo preparation, and feature work such as configurable scale questions (TEMP-351) and the admin-to-user role fix (TEMP-354). The consequence was that I often worked between people and code rather than inside one feature area. What this taught me is that a communication role is not separate from technical work. Booking the right meeting or filing the right Jira task can unblock more progress than writing one more component. What I would do differently is ask for customer prioritization earlier when the team considers adding scope.

Expectations on the team. I expected the team to collaborate and deliver a usable product. The team did that, and the weekly efficiency ratings in my progress reports averaged around 7 to 8 out of 10. But the project also showed that fullstack work based on individual tasks needs clear task descriptions and ownership. A concrete example: in the first or second sprint, two members worked on survey template logic (TEMP-185, TEMP-186, TEMP-189) with overlapping scope, which led to declined pull requests and rework. This should be understood mainly as a process problem caused by overlapping task boundaries and unclear ownership, rather than as something caused by one single person. My involvement was mainly to assist when members got stuck and help resolve merge conflicts. Later, I also experienced merge conflicts when working on the configurable scale feature (TEMP-351), but those were caused by concurrent changes in related files rather than by unclear task descriptions. My lesson is that teamwork is not only willingness to help. It is also making work small enough and visible enough that others can safely continue from it. Next time, I would push for smaller Jira tasks and clearer acceptance criteria before implementation starts, not after overlap has already happened.

Expectations on the project result. I expected a working survey system, but the result became broader than that. The project reached a survey platform supporting both web and

mobile, with admin and user flows, AI support, Qlik, Slack work, notifications, gamification, testing, and security fixes. The positive side was a richer product. The cost was more integration risk and a stronger need for prioritization. If I started again, I would ask earlier which extra features were most valuable to the customer instead of assuming that more features always meant better value.

Technical result I reached. My strongest contributions were in Cypress E2E testing, backend validation, security vulnerability handling, test environment debugging, merge conflict resolution, branch and user handling, and configurable scale questions. I also assisted with PR reviews and helped members when they got stuck.

Technical result the team and project reached. The team connected frontend, backend, database, authentication, integrations, testing, and demos into one coherent system that could be demonstrated and handed over. The project reached a broader result than a basic survey tool: a survey platform with web and mobile support, AI, Qlik analytics, Slack and email notifications, gamification, and security fixes.

Knowledge level. My knowledge improved most in automated testing, CI pipelines, test challenges related to authentication, server-side validation, and external service constraints. The team's knowledge grew because everyone had to understand more than one part of the system, from survey creation to storage, validation, analytics, and delivery.

A Acceptance Letter and Final Handoff Evidence

The acceptance letter for the final project handoff is included below. It was signed by the customer on 2026-05-18 with the outcome “Accepted and exceeding customer expectations.” The letter documents the customer’s formal acceptance of the Team Temp App delivery.

TeamTempApp

Casper Björk

casper.björk@softhouse.se

Emma Koblanck Lundmark

emma.lundmark@softhouse.se

Final delivery/Handover: 2026-05-18

The Team

Abdelraman Sandid: absn23@student.bth.se

Abduvohid Ilhomov: abil23@student.bth.se

Anas Ahmad: anbz23@student.bth.se

Anton Lindström: anlj23@student.bth.se

Filip Dennryd: fide23@student.bth.se

Filippa Lissäng Öst: fili22@student.bth.se

Kamar Haji Ali: kahj23@student.bth.se

Karam Matar: kamt23@student.bth.se

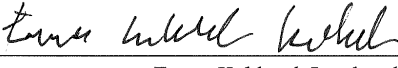
Lutfia Darouich: luda22@student.bth.se

Mouaz Allahham Naji: moap23@student.bth.se

Date of Acceptance: 18/05 - 2026

- ☒ Accepted and exceeding customer expectations
☐ Accepted, in accordance with customer expectations,
☐ Accepted, with some faults
- Explanation: _____
☐ Not accepted
- Explanation: _____


Casper Björk


Emma Koblanck Lundmark

B Scrum/Jira Board Screenshot

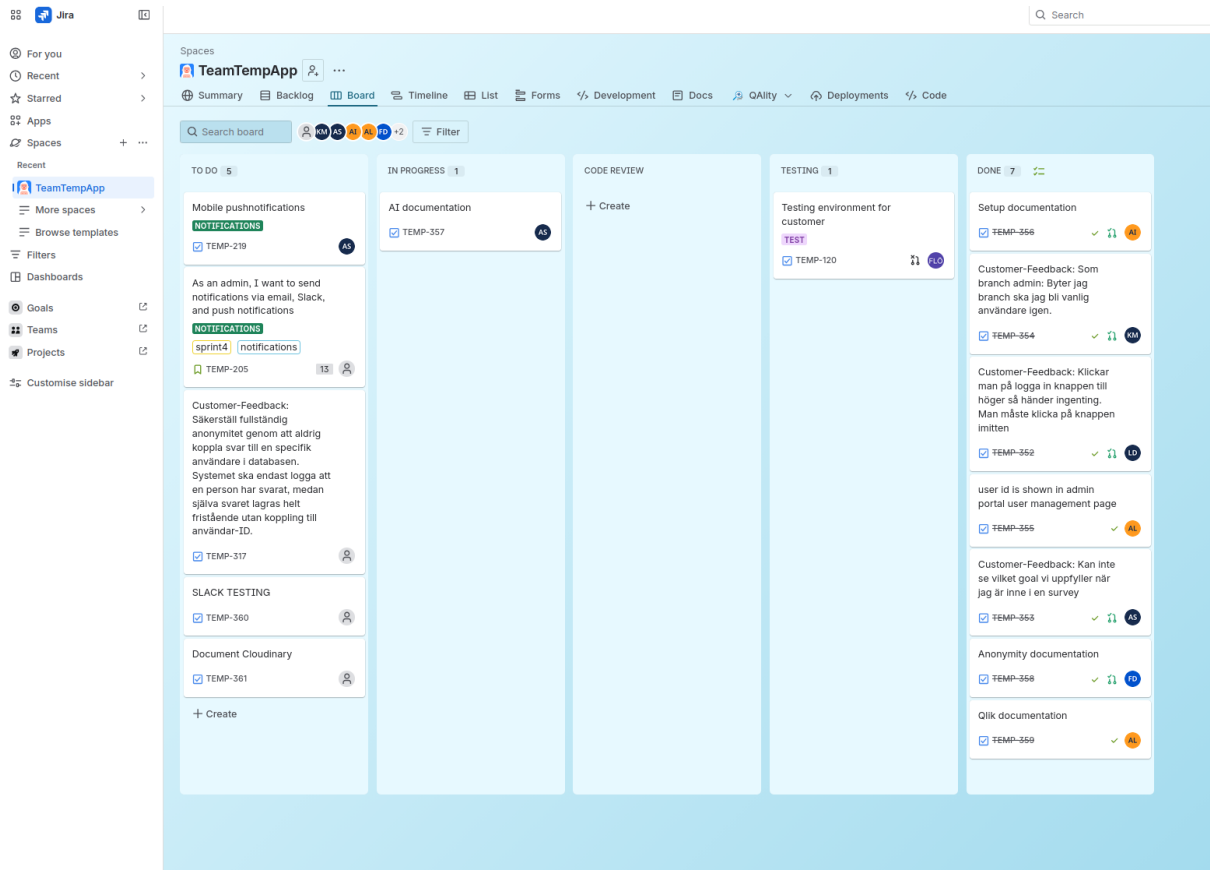


Figure 3: Final Jira board state during the polish and handoff period, with workflow columns To Do, In Progress, Code Review, Testing, and Done. Task cards show TEMP identifiers and assignee initials.

C Burn-Down/Burn-Up Chart

No burn-down or burn-up chart was actively used during the project. Progress was instead followed through Jira tasks, sprint commitments, demos, HoD follow-ups, progress reporting, and time reporting. Since no such chart was part of the team’s real process, no reconstructed chart is included here. The appendix documents this absence honestly rather than presenting a chart that was never used as a project control tool.

References

- [1] (SCRUM) *The 2020 Scrum Guide* (Ken Schwaber and Jeff Sutherland), 2020, ISBN: n/a, <https://scrumguides.org/scrum-guide.html>
- [2] (CYPRESS) *Cypress Documentation* (Cypress.io), accessed May 2026, ISBN: n/a, <https://docs.cypress.io/>
- [3] (JIRA) *What is a board in software spaces?* (Atlassian), accessed May 2026, ISBN: n/a, <https://support.atlassian.com/jira-software-cloud/docs/what-is-a-jira-software-board/>

- [4] (BITBUCKET) *Get started with Bitbucket Pipelines* (Atlassian), accessed May 2026, ISBN: n/a, <https://support.atlassian.com/bitbucket-cloud/docs/get-started-with-bitbucket-pipelines/>
- [5] (PRISMA) *Prisma Documentation* (Prisma), accessed May 2026, ISBN: n/a, <https://www.prisma.io/docs>
- [6] (REACT) *Quick Start* (React Team), accessed May 2026, ISBN: n/a, <https://react.dev/learn>